Intermediate Code Representation

Kedar Sedai

***Abstract*— The HIR(High-level intermediate Representation) phase is a 4 stage of a MiniLang compiler. After semantic analysis validates the AST, a lowering pass converts each function into a flat instruction list using temporaries, explicit local loads/stores, and labeled control flow. HIR bridges the front end and future LLVM code generation; it is the simpler than the AST for optimization, but higher-level than machine, IR, making it suitable for compiler research and debugging.**

I. INTRODUCTION

The AST is good for parsing and semantics, but not ideal for codegen/optimization. HIR acts as the bridge to LLVM. HIR Instruction is deined in *include/minilang/hir/hir.hpp*

Instruction	Meaning
`const_int / const_bool`	%t0 = 5
`load_local`	%t1 = local[n]
`store_local`	local[x] = %t0
`unary`	%t3 = - %t1
`call`	%t4 = factorial(%t5)
`ret`	return value temp
`jump/label`	gotos for if/while

TABLE I: HIR instruction design

A HirModule holds HirFunction objects, each with a flat instructions vector.

II. LOWERING AST -> HIR

HirLower in *src/hir/hir_lower.cpp* walks the AST

Expressions -> temps

Each expression becomes a fresh temp and one or more instructions:

```
AST: n <= 1
HIR: load_local %t1 = local[n]
     const_int %t2 = 1
     binary %t0 = %t1 <= %t2
```

Statements -> control flow

if (cond) { ... } else { ... }:

```
br_cond %t0 ? then_0 : else_1
label then_0:
  ...then body...
  jump endif_2
label else_1:
  ...else body...
label endif_2:
```

while (cond) {body};

```
jump while_cond_0
label while_cond_0:
  ...evaluate cond...
  br_cond %t ? while_body_1 : while_end_2
label while_body_1:
  ...body...
  jump while_cond_0
label while_end_2:
```

Scopes

- Function prameters -> locals
- Each {block} -> new scope(for future shadowing)
- declare_local tracks local names

III. RULE-BASED OPTIMIZATIONS

HirOptimizer in *src/hir/optimize.cpp* runs three passes:

A. Pass 1: Constant folding

If both operands are constants, compute at compile time:

```
const_int %t1 = 2
const_int %t2 = 3
binary %t0 = %t1 + %t2 -> const_int %t0
   = 5
```

Also folds comparisons (1 <= 1 -> true) and unary -5.

B. Pass 2: Algebraic simplification

```
x * 0 -> 0
0 * x -> 0
```

C. Pass 3: Dead temp removal

Removes computed temps whose %tN is never used (e.g. unused expression reults).

IV. DRIVER (MINILANG_HIR)

src/tools/hir_main.cpp runs the full pipeline

- Lex + parse
- Semantic analysis (must pass)
- Lower to HIR
- Optimize
- print HIR + optimization stats

Example output for factorial.minilang:

```
br_cond %t0 ? then_0 : endif_2
label then_0:
  const_int %t3 = 1
  ret %t3
...
call %t6 = factorial(%t7)
```

V. Conclusion

The HIR phase lowers the semantically valid AST into a flat, instruction-based intermediate representation with explicit temporaries, locals, and control-flow labels. Rule-based optimizations reduce constant expressions, simplify algebra, and remove and unused temporaries. Together, lowering and optimization prepare MiniLang for LLVM code generation while keeping the IR readable for research and debugging. With HIR in place, the compiler has a complete front end and middle end ready for the backend.